

ПЗ №14. Магические методы классов в Python

Тема

Изучение магических (специальных) методов в Python для настройки поведения объектов при различных операциях: сравнении, арифметических действиях, строковом представлении, итерации и других.

Цель работы

- **Теоретическая:** Изучить основные магические методы Python, их назначение и принципы использования для кастомизации поведения классов.
- **Практическая:** Приобрести навыки реализации магических методов для создания классов, которые ведут себя как встроенные типы данных (поддерживают сравнение, сложение, индексацию, итерацию и т.д.).

Задачи

1. Освоить синтаксис и применение основных магических методов.
2. Реализовать класс, использующий методы для строкового представления (`__str__`, `__repr__`).
3. Реализовать класс, поддерживающий операции сравнения (`__eq__`, `__lt__`, `__le__` и др.).
4. Реализовать класс, поддерживающий арифметические операции (`__add__`, `__sub__`, `__mul__` и др.).
5. Реализовать класс, поддерживающий итерацию (`__iter__`, `__next__`) или доступ по индексу (`__getitem__`).

Оборудование и программное обеспечение

- **Программное обеспечение:** Python 3.8+, VS Code (или любая другая среда разработки).
- **Оборудование:** Персональный компьютер.

Краткие теоретические сведения

Магические методы (или dunder-методы — от double underscore) — это специальные методы в Python, которые начинаются и заканчиваются двумя подчёркиваниями (например, `__init__`). Они позволяют определить поведение объектов для встроенных операций.

Основные группы магических методов:

1. Инициализация и представление:

- `__init__(self, ...)` – конструктор, вызывается при создании объекта
- `__str__(self)` – вызывается при преобразовании объекта в строку (`print`, `str()`)
- `__repr__(self)` – вызывается при отображении объекта в интерпретаторе, должен возвращать "официальное" строковое представление

2. Операторы сравнения:

- `__eq__(self, other)` – равенство (`==`)
- `__ne__(self, other)` – неравенство (`!=`)
- `__lt__(self, other)` – меньше (`<`)
- `__le__(self, other)` – меньше или равно (`<=`)
- `__gt__(self, other)` – больше (`>`)
- `__ge__(self, other)` – больше или равно (`>=`)

3. Арифметические операторы:

- `__add__(self, other)` – сложение (`+`)
- `__sub__(self, other)` – вычитание (`-`)
- `__mul__(self, other)` – умножение (`*`)
- `__truediv__(self, other)` – деление (`/`)
- `__floordiv__(self, other)` – целочисленное деление (`//`)

4. Операторы доступа к элементам:

- `__len__(self)` – длина объекта (`len()`)
- `__getitem__(self, key)` – доступ по индексу или ключу (`obj[key]`)
- `__setitem__(self, key, value)` – присваивание по индексу (`obj[key] = value`)
- `__delitem__(self, key)` – удаление элемента (`del obj[key]`)

5. Итерация:

- `__iter__(self)` – возвращает итератор для объекта
- `__next__(self)` – возвращает следующий элемент в итераторе

Пример реализации магических методов:

```
class Vector:
    def __init__(self, x, y):
        self.x = x
```

```

        self.y = y

    def __str__(self):
        return f"Vector({self.x}, {self.y})"

    def __repr__(self):
        return f"Vector({self.x}, {self.y})"

    def __add__(self, other):
        return Vector(self.x + other.x, self.y + other.y)

    def __eq__(self, other):
        return self.x == other.x and self.y == other.y

    def __len__(self):
        return 2 # Вектор всегда имеет 2 координаты

    def __getitem__(self, index):
        if index == 0:
            return self.x
        elif index == 1:
            return self.y
        else:
            raise IndexError("Индекс вне диапазона")

```

Порядок выполнения работы

1. Подготовительный этап

1. Запустите VS Code.
2. Создайте новый файл `lab_14_[Фамилия].py`.
3. В начале файла укажите свои данные (имя, группа) в комментарии.

2. Основной этап

Задание 1. Общий пример (выполняется всеми студентами)

Изучите приведённый ниже класс `Money`, который демонстрирует использование различных магических методов. Скопируйте его в свой файл. Создайте несколько объектов и протестируйте все операции.

```

class Money:
    """Класс, представляющий денежную сумму в рублях и копейках."""

```

```

def __init__(self, rubles, kopecks):
    self.rubles = rubles
    self.kopecks = kopecks
    self._normalize() # Приводим к нормальному виду

def _normalize(self):
    """Приводит сумму к нормальному виду (копейки от 0 до 99)."""
    if self.kopecks >= 100:
        self.rubles += self.kopecks // 100
        self.kopecks = self.kopecks % 100
    elif self.kopecks < 0:
        # Упрощённая обработка отрицательных копеек
        self.rubles += self.kopecks // 100 - 1
        self.kopecks = 100 + (self.kopecks % 100)

# Строковое представление
def __str__(self):
    """Вызывается при print() и str()."""
    return f"{self.rubles} руб. {self.kopecks:02d} коп."

def __repr__(self):
    """Вызывается в интерпретаторе."""
    return f"Money({self.rubles}, {self.kopecks})"

# Операторы сравнения
def __eq__(self, other):
    """Проверка на равенство."""
    if isinstance(other, Money):
        return (self.rubles == other.rubles and
                self.kopecks == other.kopecks)
    return False

def __lt__(self, other):
    """Меньше."""
    if isinstance(other, Money):
        self_total = self.rubles * 100 + self.kopecks
        other_total = other.rubles * 100 + other.kopecks
        return self_total < other_total
    return NotImplemented

```

```

# Арифметические операторы
def __add__(self, other):
    """Сложение."""
    if isinstance(other, Money):
        total_kopecks = (self.rubles * 100 + self.kopecks) + \
            (other.rubles * 100 + other.kopecks)
        return Money(total_kopecks // 100, total_kopecks % 100)
    return NotImplemented

def __sub__(self, other):
    """Вычитание."""
    if isinstance(other, Money):
        total_kopecks = (self.rubles * 100 + self.kopecks) - \
            (other.rubles * 100 + other.kopecks)
        if total_kopecks < 0:
            return Money(total_kopecks // 100, total_kopecks % 100)
        return Money(total_kopecks // 100, total_kopecks % 100)
    return NotImplemented

def __mul__(self, factor):
    """Умножение на число."""
    if isinstance(factor, (int, float)):
        total_kopecks = (self.rubles * 100 + self.kopecks) * factor
        return Money(int(total_kopecks // 100), int(total_kopecks %
100))
    return NotImplemented

# Демонстрация работы
print("Создаём объекты:")
m1 = Money(10, 50)
m2 = Money(5, 75)
m3 = Money(10, 50)

print(f"m1 = {m1}")          # __str__
print(f"m2 = {m2}")
print(f"m3 = {m3}")

print("\nСравнение:")
print(f"m1 == m3: {m1 == m3}") # __eq__
print(f"m1 == m2: {m1 == m2}")
print(f"m1 > m2: {m1 > m2}")   # __gt__ автоматически генерируется из

```

```

__lt__
print(f"m1 < m2: {m1 < m2}")      # __lt__

print("\nАрифметика:")
print(f"m1 + m2 = {m1 + m2}")      # __add__
print(f"m1 - m2 = {m1 - m2}")      # __sub__
print(f"m1 * 3 = {m1 * 3}")        # __mul__

print("\nОтрицательные суммы:")
m4 = Money(0, 50) - Money(1, 0)
print(f"0.50 - 1.00 = {m4}")      # Отрицательные рубли

```

Что нужно сделать: Запустите этот код, изучите поведение каждого магического метода. Обратите внимание, что для полной поддержки всех сравнений достаточно реализовать `__eq__` и `__lt__` – остальные методы Python может сгенерировать автоматически.

Задание 2. Самостоятельная разработка классов по вариантам

Выберите свой вариант и реализуйте описанный класс, используя магические методы. Каждый класс должен содержать:

Обязательные магические методы для всех вариантов:

- `__init__` – конструктор с параметрами
- `__str__` – красивое строковое представление
- `__repr__` – официальное строковое представление

Дополнительные магические методы в зависимости от варианта:

- методы сравнения (`__eq__`, `__lt__`, `__le__` и др.)
- методы арифметических операций (`__add__`, `__sub__`, `__mul__` и др.)
- методы доступа по индексу (`__len__`, `__getitem__`, `__setitem__`)
- методы итерации (`__iter__`, `__next__`)

Вариант 1. Класс `Fraction` (Дробь)

Реализуйте класс для работы с обыкновенными дробями (числитель и знаменатель).

- Конструктор принимает числитель и знаменатель (знаменатель не может быть 0)
- `__str__` возвращает строку вида "числитель/знаменатель"
- `__repr__` возвращает строку вида "Fraction(числитель, знаменатель)"
- `__add__` – сложение двух дробей
- `__sub__` – вычитание дробей

- `__mul__` – умножение дробей
- `__truediv__` – деление дробей
- `__eq__` – сравнение на равенство
- `__float__` – возвращает десятичное представление дроби

Вариант 2. Класс `BookShelf` (Книжная полка)

Реализуйте класс, представляющий книжную полку со списком книг.

- Конструктор принимает список книг (строк) или создаёт пустую полку
- `__len__` – возвращает количество книг на полке
- `__getitem__` – возвращает книгу по индексу
- `__setitem__` – устанавливает книгу по индексу
- `__iter__` – позволяет итерироваться по книгам
- `__add__` – объединяет две книжные полки (возвращает новую полку)
- `__contains__` – проверяет, есть ли книга на полке (`in`)
- `__str__` – выводит список книг в удобном виде

Вариант 3. Класс `Temperature` (Температура)

Реализуйте класс для работы с температурой в градусах Цельсия.

- Конструктор принимает значение температуры в Цельсиях
- `__str__` возвращает строку вида "25.5°C"
- `__repr__` возвращает строку вида "Temperature(25.5)"
- `__add__` – сложение температур (возвращает новый объект)
- `__sub__` – вычитание температур
- `__eq__`, `__lt__`, `__gt__` – все сравнения температур
- `__int__` – возвращает температуру в виде целого числа
- `__float__` – возвращает температуру как float
- Дополнительные методы: `to_fahrenheit()` – переводит в Фаренгейты, `to_kelvin()` – в Кельвины

Вариант 4. Класс `Matrix` (Матрица 2x2)

Реализуйте класс для работы с матрицей 2x2.

- Конструктор принимает 4 числа: a, b, c, d (элементы матрицы `a, b`], `[c, d]`)
- `__str__` выводит матрицу в виде:

```
[a b]
[c d]
```

- `__repr__` возвращает строку вида "Matrix(a, b, c, d)"
- `__add__` – сложение матриц (поэлементно)
- `__sub__` – вычитание матриц

- `__mul__` – умножение матриц
- `__matmul__` – умножение матриц через оператор `@`
- `__eq__` – сравнение матриц
- `__getitem__` – доступ по индексу: `matrix[0][1]` возвращает элемент (0,1)
- `determinant()` – вычисляет определитель матрицы

Вариант 5. Класс `StudentGroup` (Группа студентов)

Реализуйте класс для работы с группой студентов (список оценок).

- Конструктор принимает название группы и список оценок студентов (список целых чисел от 2 до 5)
- `__len__` – возвращает количество студентов
- `__getitem__` – возвращает оценку студента по индексу
- `__setitem__` – изменяет оценку студента с проверкой (от 2 до 5)
- `__iter__` – позволяет итерироваться по оценкам
- `__contains__` – проверяет, есть ли определённая оценка в группе
- `__add__` – объединяет две группы (возвращает новую группу)
- `__str__` – выводит название группы и список оценок
- `average()` – вычисляет средний балл группы
- `excellent()` – возвращает количество отличников (оценки 5)

Задание 3. Проверка работоспособности

Для своего варианта:

1. Создайте **минимум 4 объекта** с разными параметрами.
2. Продемонстрируйте работу **всех реализованных магических методов**.
3. Покажите работу дополнительных методов (если они есть).
4. Вывод должен быть наглядным и сопровождаться пояснениями.

Пример оформления проверки (для варианта 1):

```
print("=" * 50)
print("Тестирование класса Fraction")
print("=" * 50)

# Создание объектов
frac1 = Fraction(1, 2)
frac2 = Fraction(1, 3)
frac3 = Fraction(2, 4) # Должна быть равна 1/2

print(f"frac1 = {frac1}")
```



```

print(f"frac2 = {frac2}")
print(f"frac3 = {frac3}")

print("\nАрифметические операции:")
print(f"{frac1} + {frac2} = {frac1 + frac2}")
print(f"{frac1} - {frac2} = {frac1 - frac2}")
print(f"{frac1} * {frac2} = {frac1 * frac2}")
print(f"{frac1} / {frac2} = {frac1 / frac2}")

print("\nСравнение:")
print(f"{frac1} == {frac3}: {frac1 == frac3}")
print(f"{frac1} > {frac2}: {frac1 > frac2}")

print("\nПреобразование в float:")
print(f"{frac1} = {float(frac1)}")
print(f"{frac2} = {float(frac2)}")

# Проверка обработки некорректных данных
try:
    frac_error = Fraction(1, 0)
except ValueError as e:
    print(f"\nОшибка при создании дроби: {e}")

```

3. Контрольный этап

1. Проверьте, что все реализованные магические методы работают корректно.
2. Убедитесь, что в методах есть обработка некорректных типов данных (например, при сложении с объектом другого класса возвращается `NotImplemented` или вызывается исключение).
3. Проверьте, что классы корректно обрабатывают пограничные случаи (нули, отрицательные числа, пустые коллекции и т.д.).

Контрольные вопросы

1. Что такое магические методы в Python? Почему они так называются?
2. Какие магические методы отвечают за строковое представление объекта? В чём разница между `__str__` и `__repr__`?
3. Какие методы нужно реализовать, чтобы объект поддерживал операции сравнения (`>`, `<`, `==` и др.)?
4. Для чего используется метод `__call__`? Приведите пример.
5. Как сделать объект итерируемым (поддерживающим цикл `for`)?

6. Какие методы позволяют работать с объектом как с коллекцией (доступ по индексу, определение длины)?
7. Что такое `NotImplemented` и зачем его возвращать из арифметических методов?
8. Можно ли перегрузить оператор `+` для сложения объекта вашего класса с объектом встроенного типа (например, `int`)?
9. Какой магический метод вызывается при использовании `with` (контекстный менеджер)?
10. Почему в Python не используются перегрузки операторов так же, как в C++ или Java? Каковы особенности?

Содержание отчета

1. Тема, цель и задачи работы.
2. Код программы (включая общий пример и реализацию своего варианта) с комментариями.
3. Результаты выполнения (вывод программы – скриншот или текст).
4. Ответы на контрольные вопросы.
5. **Выводы по работе:** Что нового узнали о магических методах? Как они расширяют возможности классов? Какие магические методы показались наиболее полезными? Удалось ли достичь цели работы? Какие трудности возникли?